Kassidy Shomo

Professor Kraya

CS 499: Computer Science Capstone

June 18, 2026

Enhancement Three: Databases

The artifact selected for the databases category was originally developed on January 13 as part of CS 405: Secure Coding. The program was designed to demonstrate how SQL injection vulnerabilities can occur when user input is incorporated directly into SQL statements. The artifact uses an in-memory SQLite database to store user records and execute queries, providing a practical example of how database applications retrieve and process data. The original version focused on identifying and blocking common SQL injection attempts through regular expression pattern matching after a query had already been constructed.

I selected this artifact for my ePortfolio because it demonstrates both database programming and secure software development practices. The program showcases my ability to work with relational databases, execute queries, retrieve and process data, and implement security-focused solutions within a database-driven application. It also provided a strong opportunity for enhancement because the original implementation relied on detecting suspicious query patterns rather than preventing SQL injection through secure database design. This allowed me to demonstrate growth by redesigning how the application interacts with the database instead of simply adding additional validation logic.

In the original version of the artifact, SQL injection prevention relied on analyzing completed SQL statements with a regular expression before execution. The program attempted to identify patterns such as OR 1=1 or similar expressions commonly associated with injection attacks. While this approach demonstrated awareness of SQL injection risks, it had limitations because it depended on detecting known patterns after the query had already been created. This meant the solution focused on identifying potentially dangerous input rather than preventing malicious input from influencing query structure in the first place.

The enhanced version restructures database interaction by replacing the regular expression detection mechanism with prepared statements and parameterized queries. Instead of directly incorporating user input into SQL commands, the enhanced implementation separates SQL structure from user-supplied values and binds those values as parameters before execution. This ensures that user input is treated strictly as data and cannot alter the logic of the SQL statement.

To support this change, the enhanced version introduces the reusable function execute_safe_query, which centralizes query preparation, parameter binding, execution, and result retrieval. A second function, get_user_by_name, further separates database access logic from the rest of the application by providing a dedicated method for retrieving records. This creates a clearer separation of responsibilities between database operations and program control flow while improving maintainability and readability.

The differences between the original and enhanced implementation can be summarized below:

Original	Enhanced
Regular expression pattern matching used to detect injection attempts	Prepared statements and parameter binding prevent injection attempts

SQL analyzed after query construction	SQL structure separated from user input before execution
Detection-based security approach	Prevention-based security approach
Database logic handled through direct query execution	Reusable database execution functions
Limited protection against unknown attack patterns	Consistent protection through parameterized queries

The enhanced version also preserves the educational purpose of the original artifact by testing multiple SQL injection payloads, including common authentication bypass attempts and a simulated DROP TABLE attack. Unlike the original version, these inputs are passed into prepared statements as bound parameters, demonstrating that they are treated as ordinary data rather than executable SQL commands.

This enhancement aligns strongly with the databases category because it focuses directly on improving how the application interacts with a relational database. The enhancement demonstrates database security practices, query execution techniques, parameter binding, data retrieval, and structured database access design. It also reflects real-world industry practices used to secure database-driven applications against common attack vectors.

This enhancement aligns with Course Outcomes 3, 4, and 5. It aligns with Outcome 3 because it demonstrates the design and evaluation of a computing solution that improves database security while balancing security, usability, and maintainability. It aligns with Outcome 4 through the implementation of industry-standard database techniques such as prepared statements and parameterized queries to improve how data is accessed and managed within the application. It aligns with Outcome 5 by applying a security-focused mindset to identify weaknesses in the original design and replace them with a more reliable approach for preventing SQL injection

attacks. At this time, no updates to my outcome coverage plans are necessary because the enhancement addresses the outcomes originally identified in the proposal.

Through the enhancement process, I gained a deeper understanding of how database security is most effective when vulnerabilities are prevented through design rather than detected after the fact. While the original implementation attempted to identify suspicious patterns within SQL statements, the enhancement demonstrated that secure query construction is a more reliable approach because it removes the opportunity for malicious input to alter query logic. I also learned more about SQLite prepared statements, parameter binding, and the importance of separating responsibilities within a program to improve maintainability. One challenge I encountered was restructuring the application while preserving the original educational purpose of demonstrating SQL injection concepts. I needed to redesign the database interaction model without losing the ability to illustrate how malicious input behaves when passed into a secure system. Working through that challenge strengthened both my understanding of database security and my ability to refactor existing code into a more secure and maintainable solution.